

INTRODUCTION

Compiler Design

- Introduction of and various phases of compiler Design :-

→ The main aim of compiler design is to convert a pure high level language into low level language.

HLL (high level language)

↓
[pre-processor]

↓ Pure HLL

[Compiler]

↓ assembly language

[Assembler]

↓ m/c code (relocatable)

[loader/linker]

↓
[executable code/
absolute m/c code]

{<#include> - file inclusion

{<#define> - macro expansion

↓ (if any language contain this type of line it called HLL)

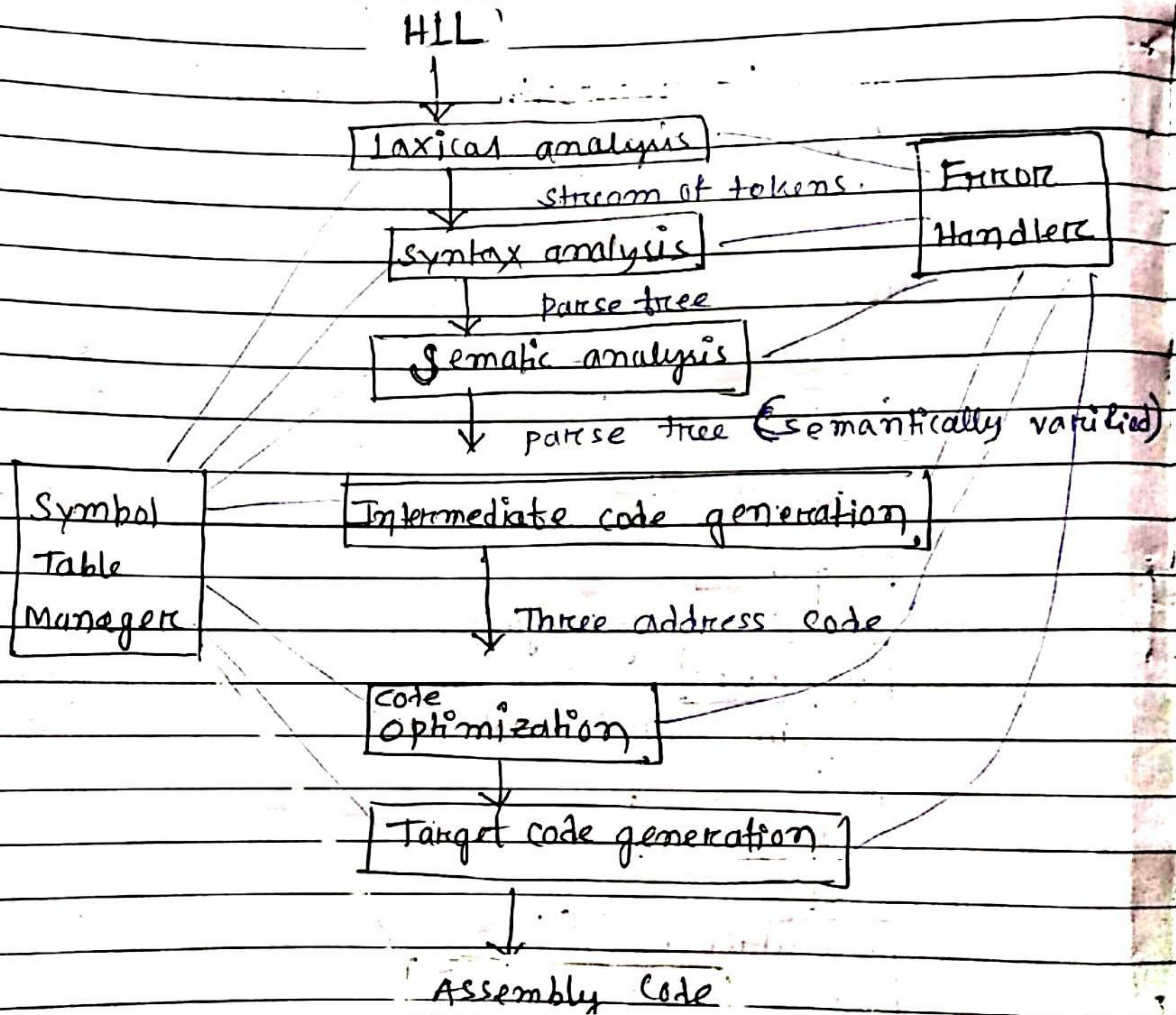
→ pure HLL means program not contain any '#' line.

→ preprocessor : is responsible for

1. Macro expansion
2. File inclusion.

→ loader : is responsible for

1. Allocation.
2. Re-allocation.
3. Linking.
4. Loading.

Compiler phases

Lex and yacc : are tools used in Unix operating system for compiler design. First compiler is FORTRAN. It took 18 years to build it.

Lexical analysis :

HLL text or source text is broken into tokens.

ex: $\text{If } (A > B) \rightarrow 10 \text{ tokens.}$
 $a = 10;$

Example of all the phases of compiler —

$x = a + b * c ; \rightarrow$ source program.

Lexical analyser

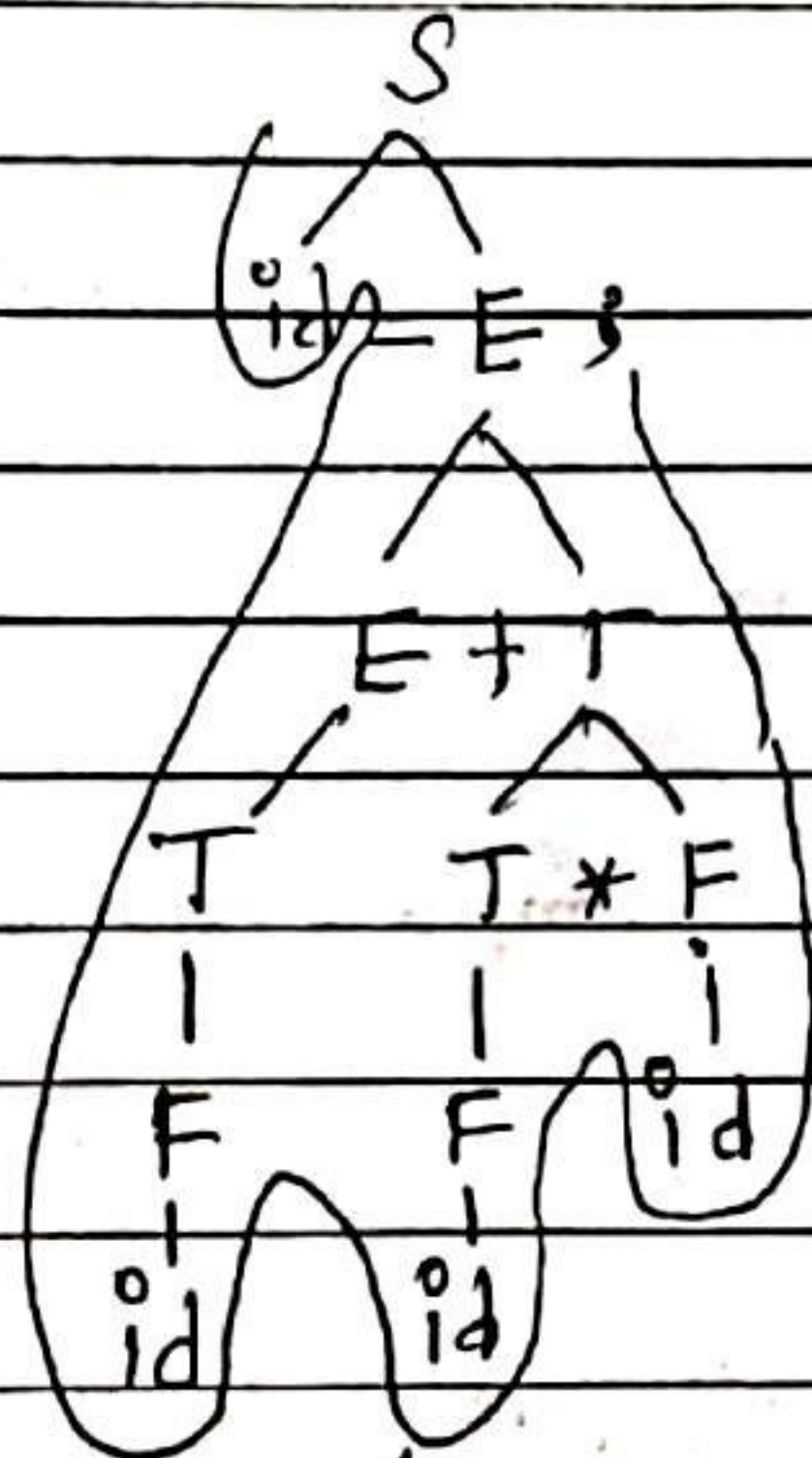
$id = id + id * id$

$\rightarrow (id = identifier)$

heart of the
Compiler

Syntax analyser

$S \rightarrow id = E ;$	(CFG)
$E \rightarrow E + T / T$	
$T \rightarrow T * F / F$	
$F \rightarrow id$	



Semantic analyser

(parse tree semantically
verified)

ICG

frontend

$t_1 = b * c$

$t_2 = a + t_1$

$x = t_2$

Code optimization

$t_1 = b * c$
 $x = a + t_1$

↓
Target code generation
 ↓

{	mul R ₁ ; R ₂	a → R ₀
	add R ₀ , R ₂	b → R ₁
	mov R ₂ , M	c → R ₂

→ Assembly code. ✓ Backend.

→ Lex(tool) used to implement 'Lexical analyser'.

→ yacc(tool) " " " 'Syntax analyser'.

→ practically we have two phases of compiler — frontend and Backend.

→ To do any project on compiler have tool called 'LANCE'.

• Lexical analyser:

```

      ↗ Lexemes ↘
int max(x,y)
int x,y;
/* find max of x and y */
{
    return (x > y ? x : y);
}
  
```

→ Lexical analyser removing the comments and all white spaces.

→ main function of lexical analyser is converting Lexemes into tokens.

→ Show the error. (if getting any error).

Ex: How many token is there in this particular lines =

```

① int/ max/(x,y)/
   int/ u/y;/
   /* find max of x and y */
   {/
   return/(u>y?u:y)/;/
   }/
→ 25 (tokens)

```

```

② printf/(" %d Hai", &u);/
→ 8 (tokens are there)

```

→ Question can come from lexical analyser is
(How many token is there and what are the responsibility)

• Grammars:

Input
②
Syntax analyzer ← ① Grammar

$G = (V, T, P, S)$

$V \rightarrow$ variable

$T \rightarrow$ Terminal

$P \rightarrow$ Production

$S \rightarrow$ start symbol.

Ex: $(E) \rightarrow E + E$
 $\quad \quad \quad / E * E$
 $\quad \quad \quad / id$

here,

$V = \{E\}$

$T = \{+, *, id\}$

start symbol = E.

(id + id * id) generate this string using given rule (LMD)
Left most derivation = (LMD) Right most derivation =

$E \Rightarrow E + E$
 $\Rightarrow id + E$
 $\Rightarrow id + E * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$

$E \Rightarrow E + E$
 $\Rightarrow E + E * E$
 $\Rightarrow E + E * id$
 $\Rightarrow E + id * id$
 $\Rightarrow id + id * id$

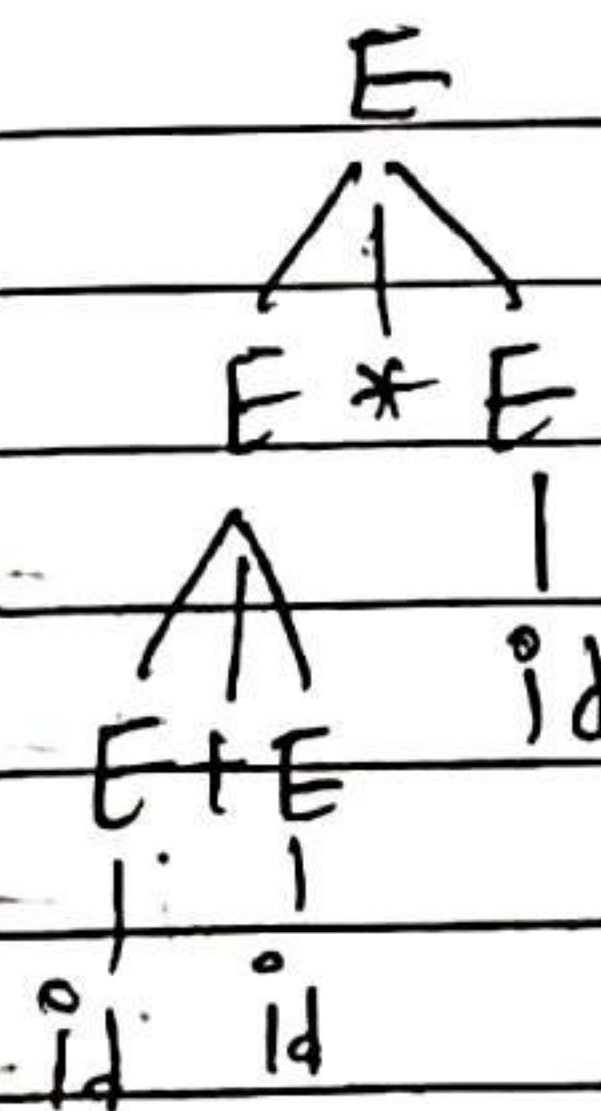
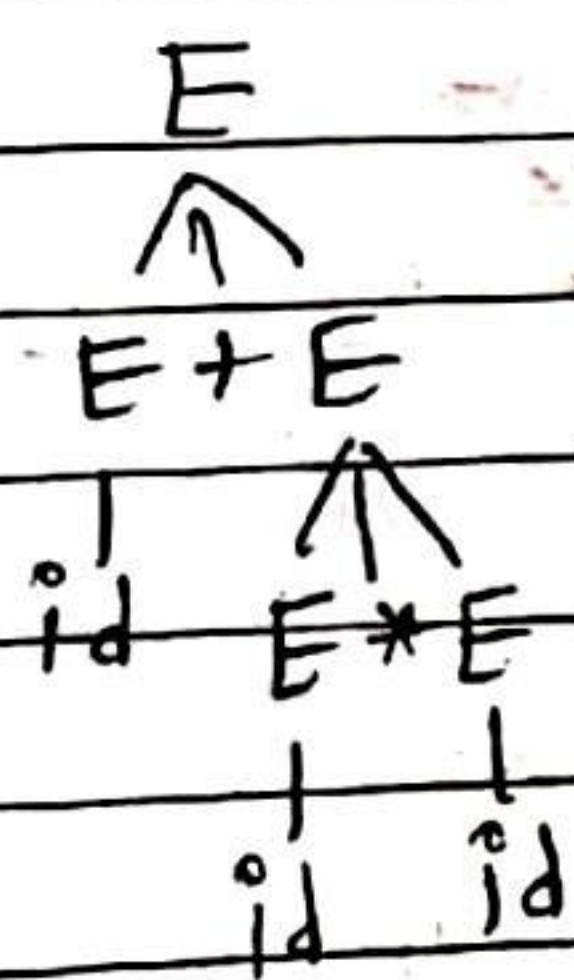
LMD =

RMD =

$E \Rightarrow E * E$
 $\Rightarrow E + E * E$
 $\Rightarrow id + E * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$

$E \Rightarrow E * E$
 $\Rightarrow E * id$
 $\Rightarrow E + E * id$
 $\Rightarrow E + id * id$
 $\Rightarrow id + id * id$

• Derived using parse tree -



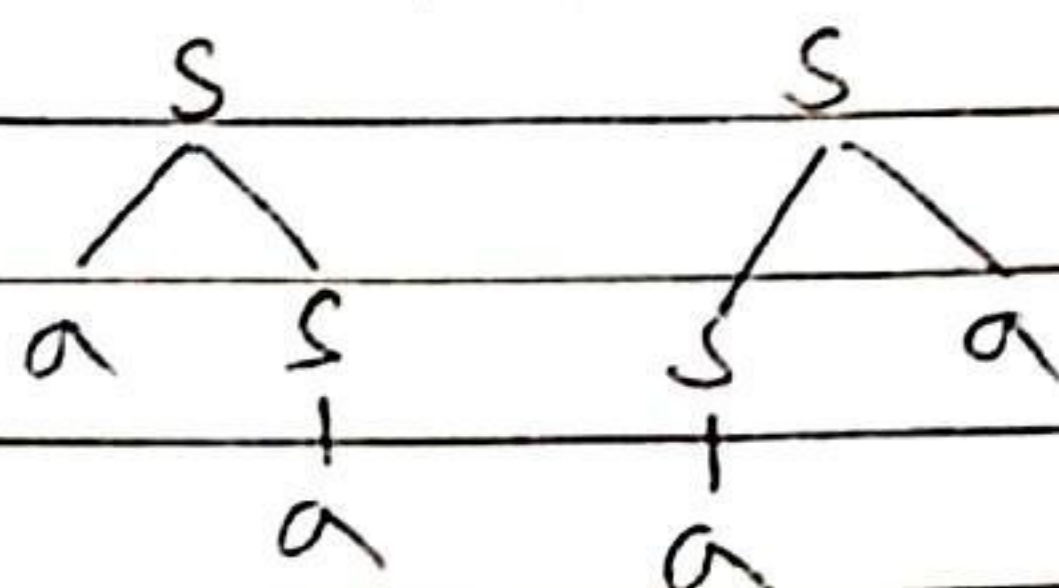
→ To the given grammar and string if we get more than one LMD or RMD or parse tree then the grammar is called ambiguous grammar.

→ ambiguity problem is undecidable.
(because no algo to solve the problem)

Find given grammars are ambiguous or not:

① $S \rightarrow aS / Sa / a$

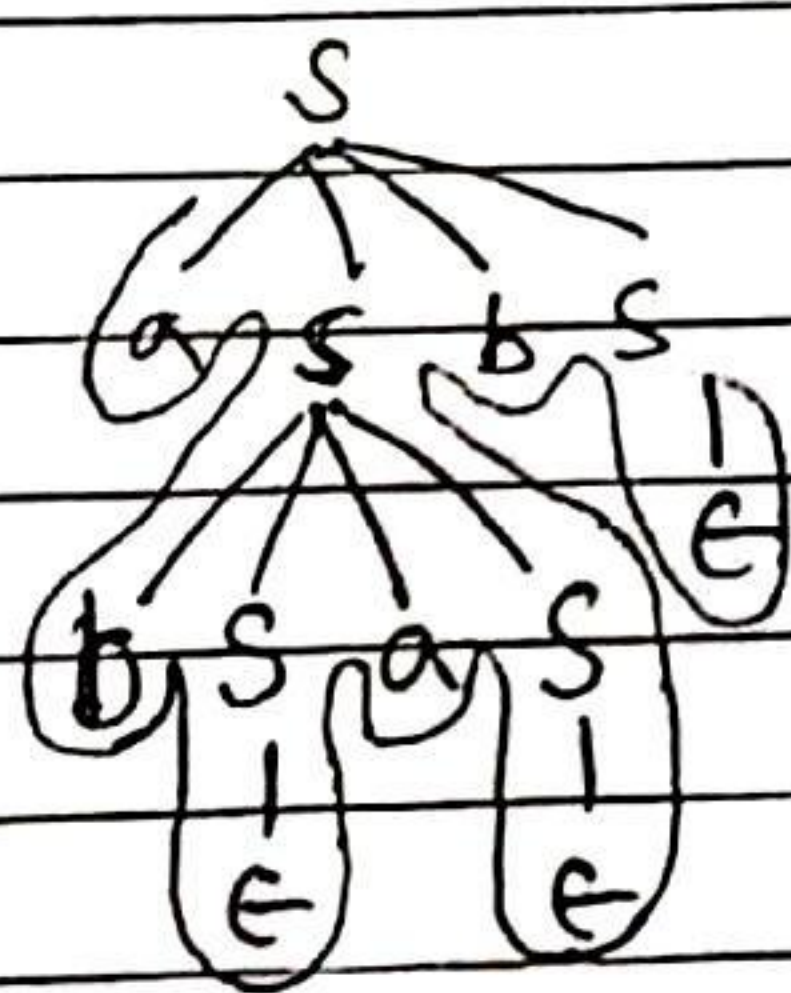
→ $w = aa$



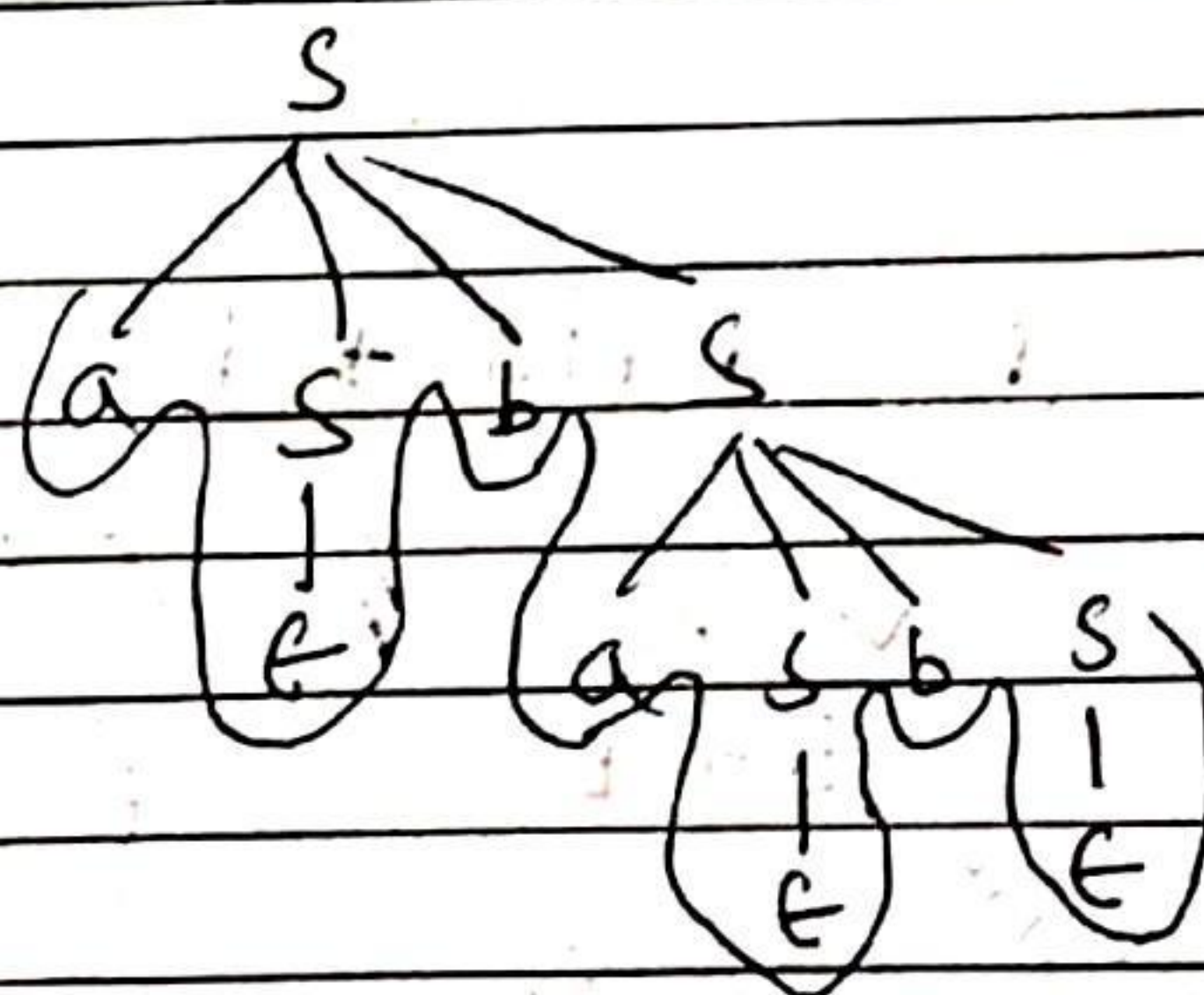
→ This grammar is ambiguous.

② $S \rightarrow aSbS / bSaS / \epsilon$

→ $w = abab$



(abab)



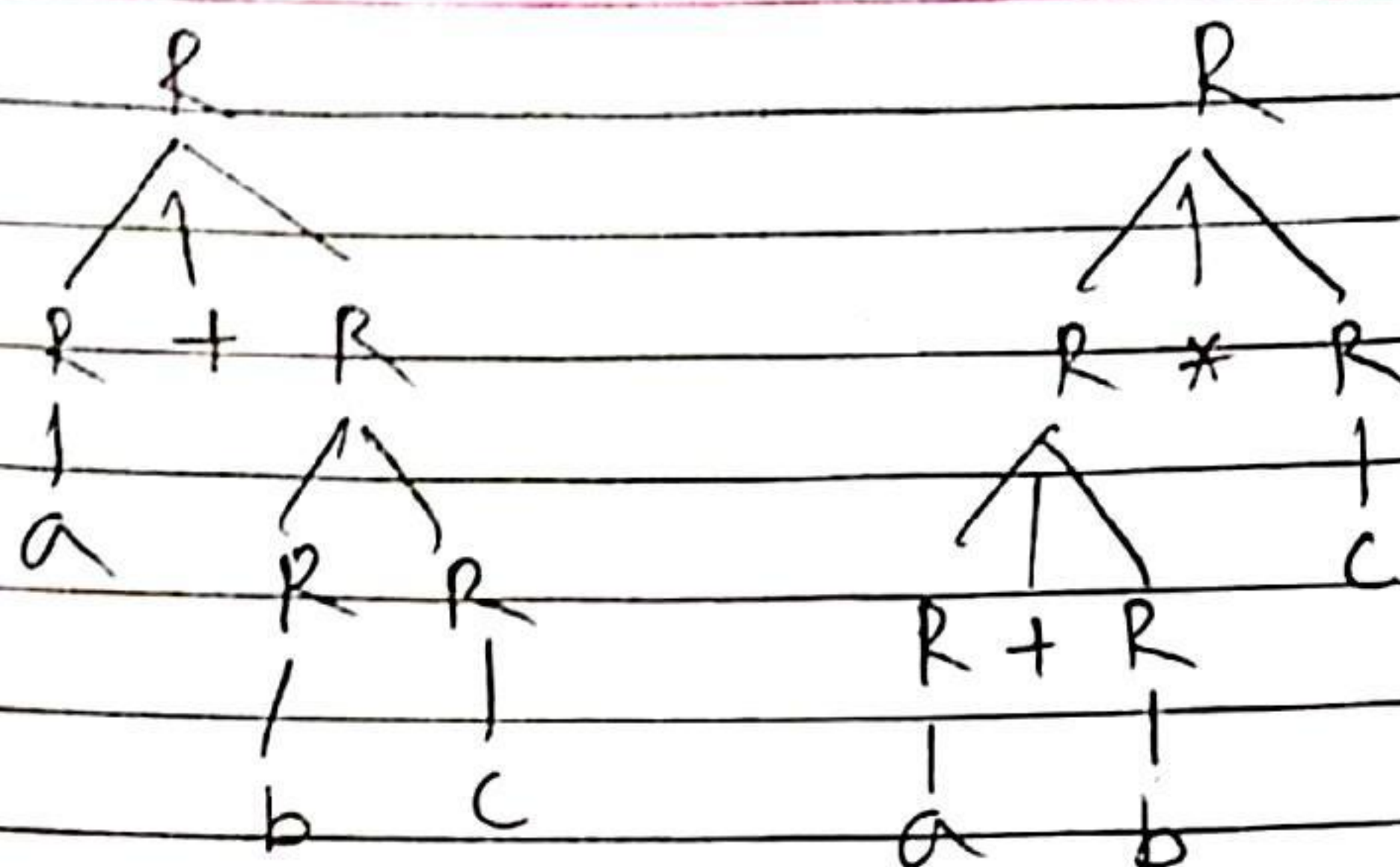
(abab)

→ To the same string given grammar and string we get more than one parse tree, so that the given grammar is ambiguous.

③

$R \rightarrow R+R / RR / R^* / a / b / c$

→ $w = a+bc$



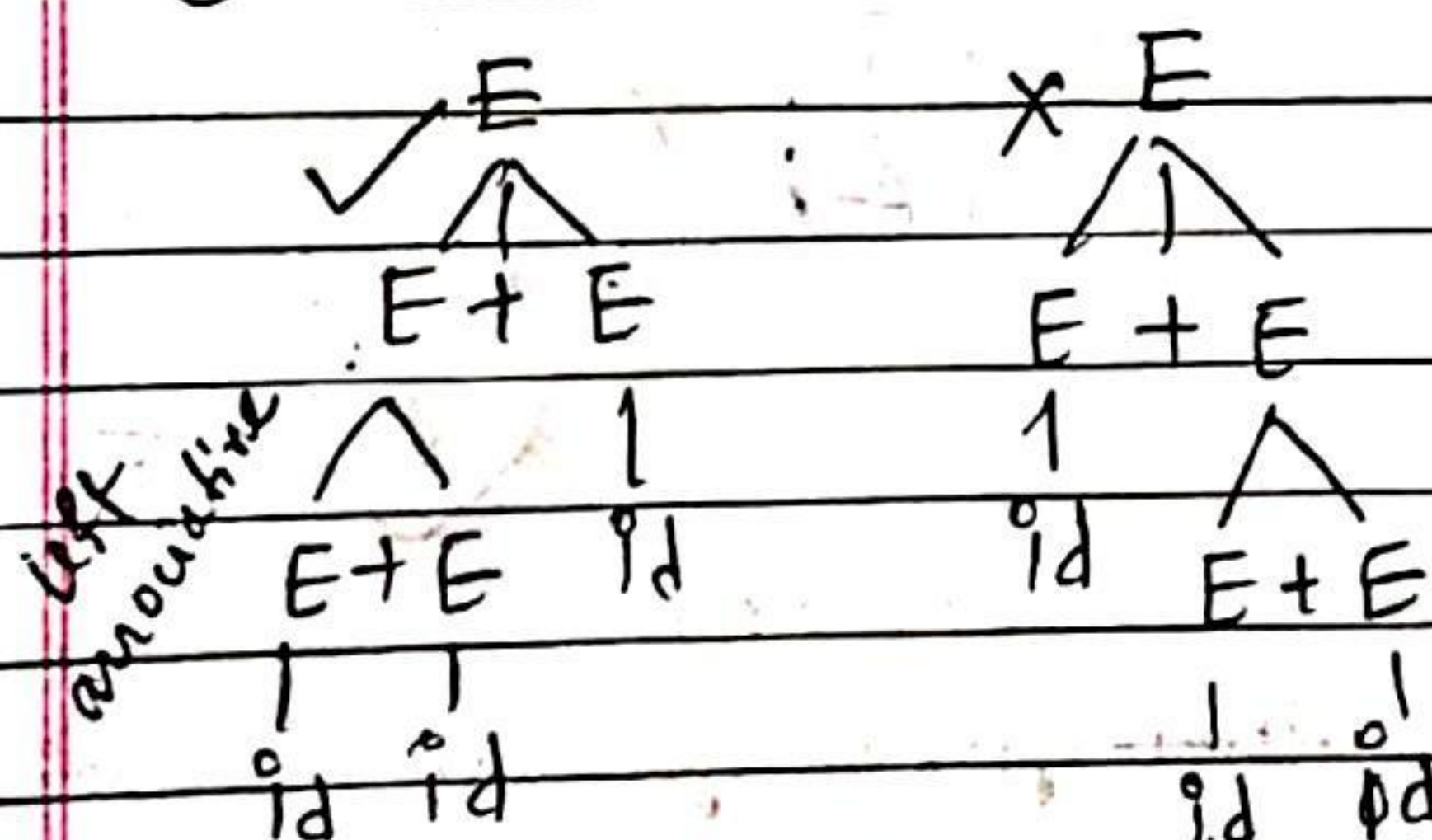
→ this grammar is ambiguous grammar.

- Ambiguous grammar and making them unambiguous:

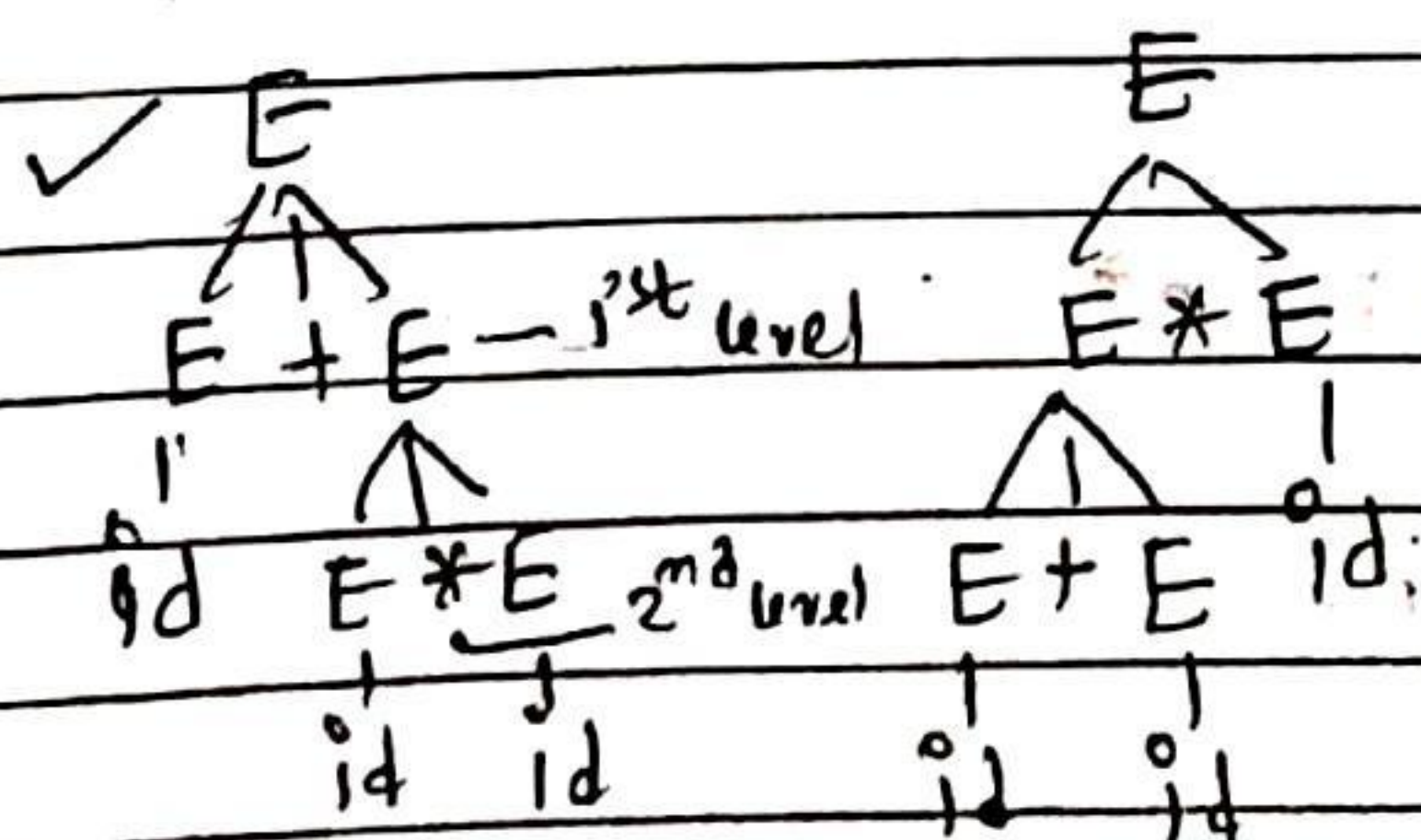
Ex-1

$E \rightarrow E + E$
 $\quad \quad \quad / E * E$
 $\quad \quad \quad / id$

① $id + id * id$

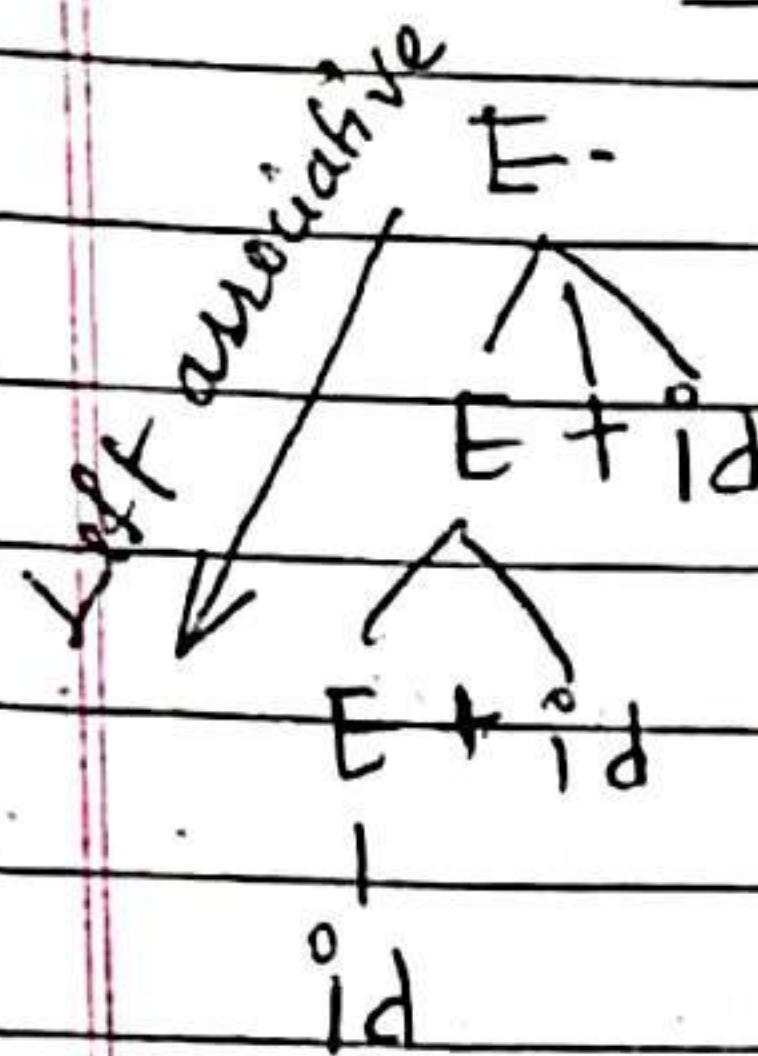


② $id + id * id$



Let operators are left associative.
~~Let operators are right associative.~~

① $E \rightarrow E + id / id$ (this grammar is left recursive)



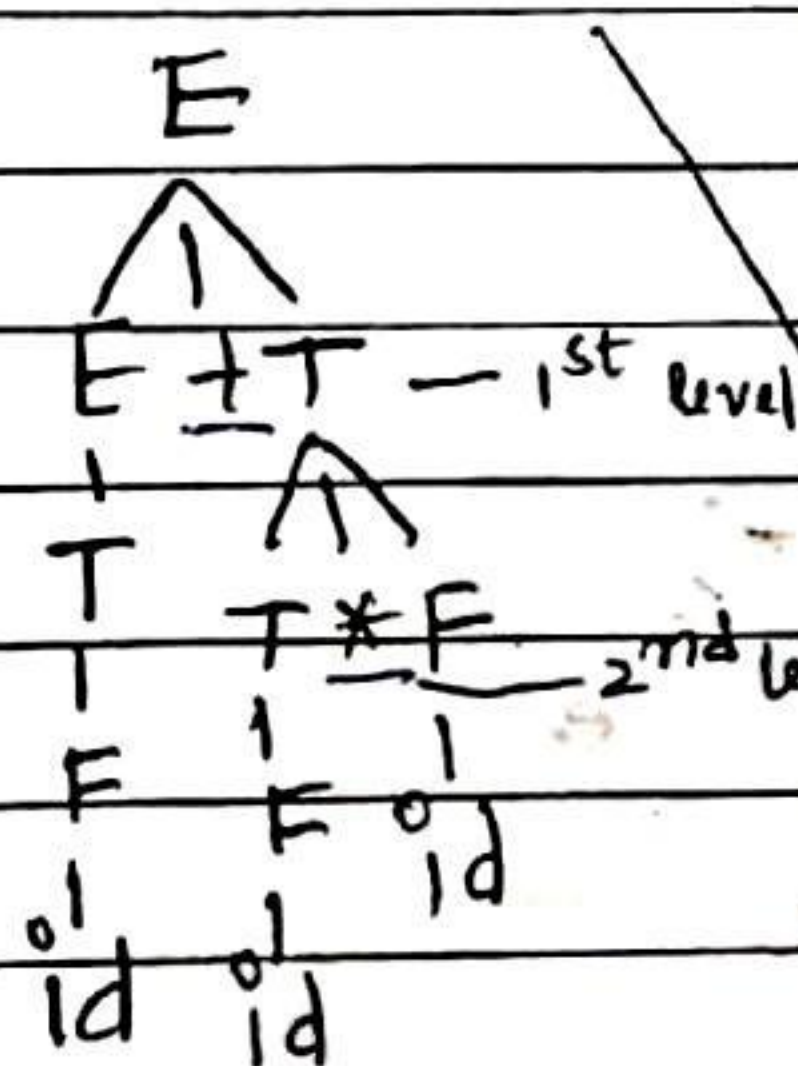
$(id + id) + id$

Now the grammar is unambiguous.

② $E \rightarrow E + T / T$

$T \rightarrow T * F / F$

$F \rightarrow id$



$(id + (id * id))$

Now the grammar is unambiguous.

$$(2^{(3^2)}) = 2 \uparrow 3 \uparrow 2$$

precedence: $\uparrow > * > +$
 3rd level 2nd level 1st level

③ $E \rightarrow E + T / T$

$T \rightarrow T * F / F$

$F \rightarrow G \uparrow F / G$

$G \rightarrow id$

here, + and * is left associative,

$\uparrow$ is Right associative.

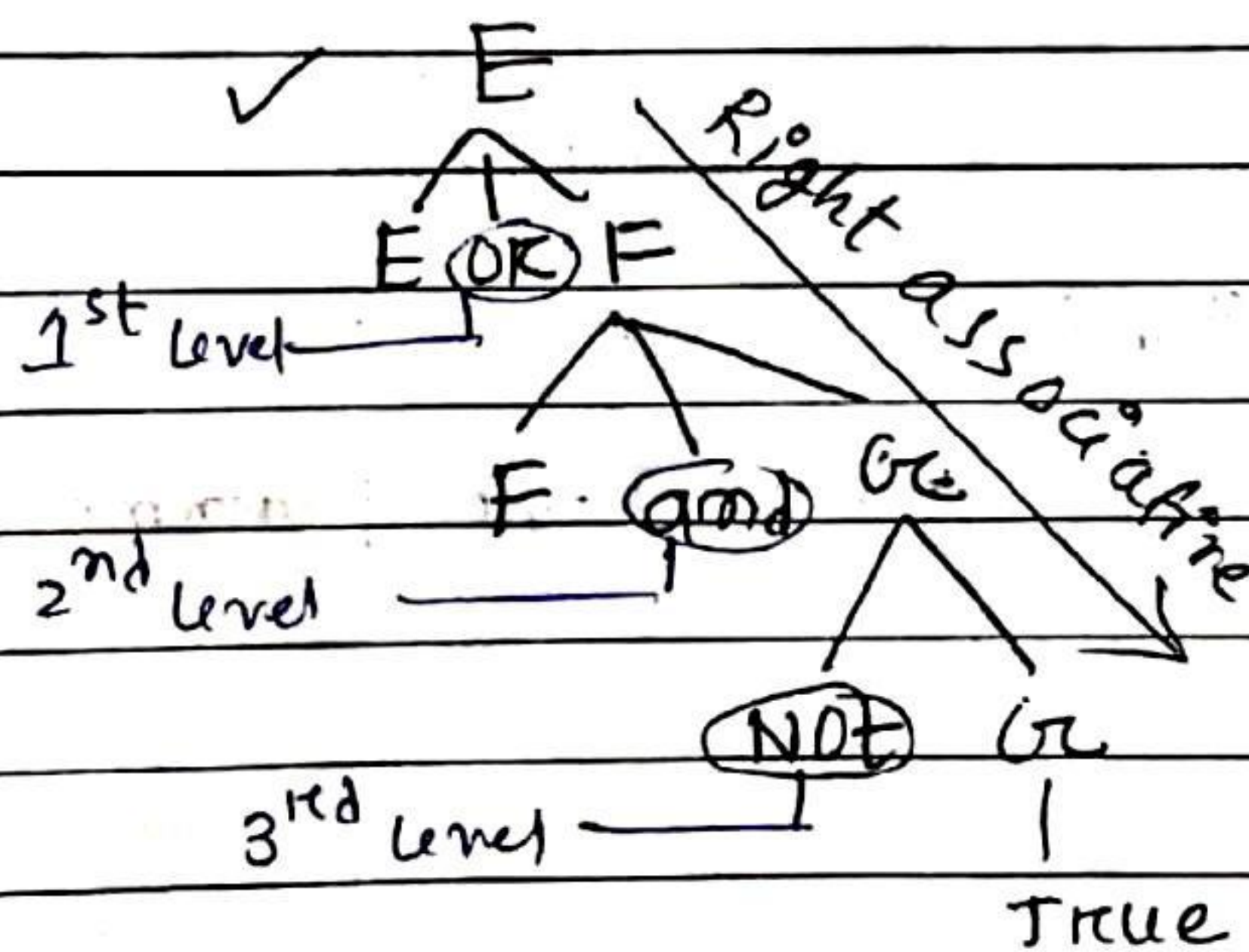
④ boolean expression,

convert

$(b \text{ Exp}) \rightarrow (b \text{ Exp}) \text{ OR } (b \text{ Exp})$
 $/ (b \text{ Exp}) \text{ and } (b \text{ Exp})$
 $/ \text{ not } (b \text{ Exp})$
 $/ \text{ True}$
 $/ \text{ False}$

$(E) \rightarrow (E) \text{ OR } F / F$
 $(F) \rightarrow (F) \text{ and } G / G$
 $(G) \rightarrow \text{Not } (G) / \text{True} / \text{False}$

→ This grammar is unambiguous, because lower precedence operators closest to the start symbol (1st level) and higher precedence operators are least level.
 → And this grammar follows the Associativity rule.



⑤ $R \rightarrow R + R$ (convert into unambiguous grammar)

$/ RR$

$/ R^*$

$/ a$

$/ b$

$/ c$

$E \rightarrow E + T / T$

$T \rightarrow T * E / E$

$F \rightarrow F * / a / b / c$

→ Now this grammar is unambiguous grammar.

⑥ Given grammar -

$$A \rightarrow A \$ B / B$$

(\$, #, @ are operators)

$$B \rightarrow B \# C / C$$

$$C \rightarrow C @ D / D$$

(here, \$, #, @ are left recursive)

$$D \rightarrow d$$

$$\rightarrow \$ \triangleright \$$$

$$\# \triangleright \#$$

$$@ \triangleright \#$$

$$a \triangleright a$$

higher precedence

$$\rightarrow \$ \prec \# \prec @ \text{ Operators are evaluated.}$$

⑦ Given grammar -

$$E \rightarrow (E * F) \rightarrow \text{here, } * \text{ is left associative. (left recur)}$$

$$/ F + (E) \rightarrow + \text{ is Right associative. (right recur)}$$

$$/ F$$

$$F \rightarrow (E - E) \rightarrow \text{Associativity is undefined. because, here, } - \text{ define left or Right associative both.}$$

$$/ id$$

→ And This grammar is ambiguous.

$$* \hat{=} + \text{ (equal)}$$

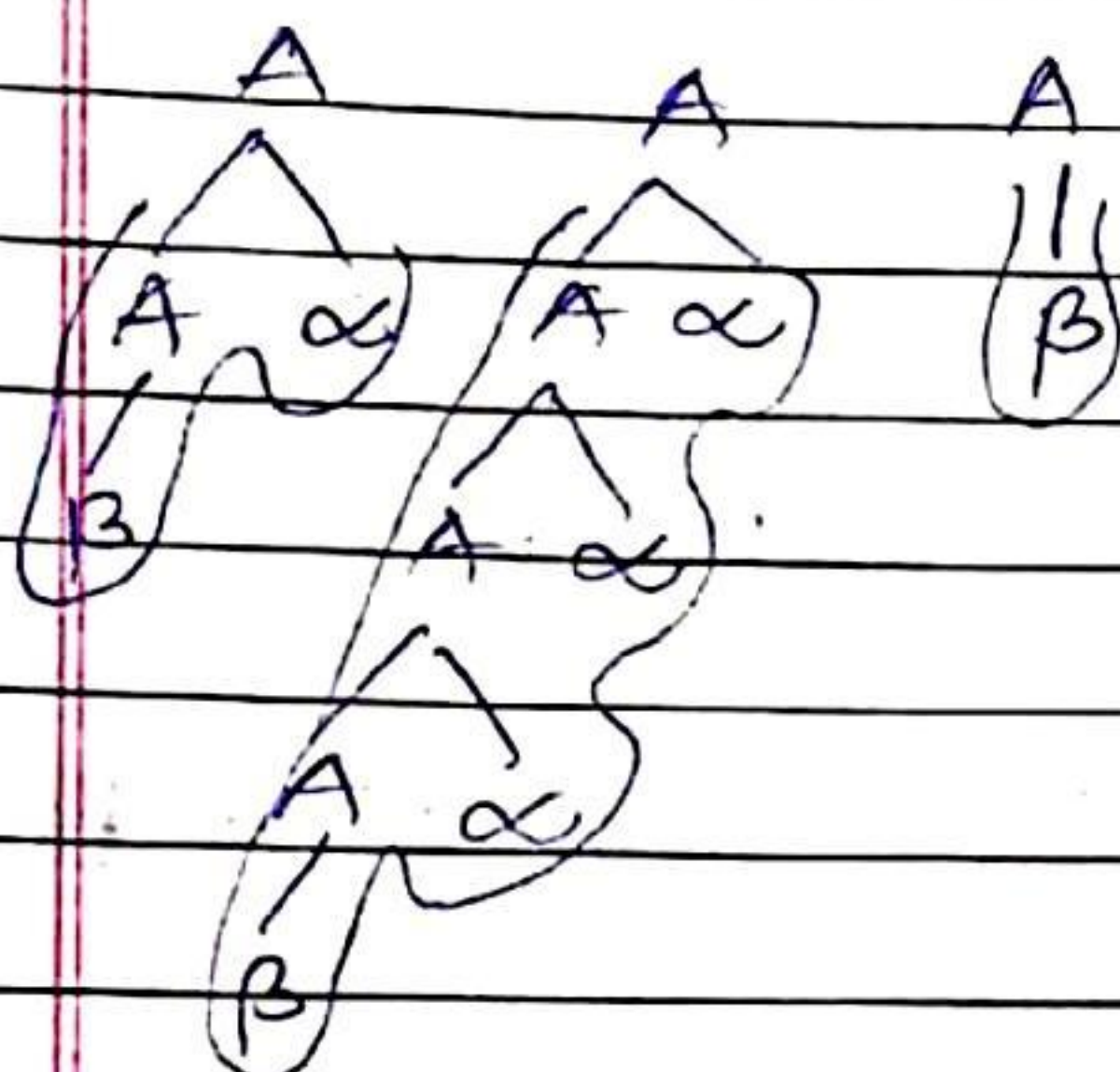
$$\text{(high precedence)} * \triangleright *$$

$$+ \prec + \text{ (high precedence)}$$

Recursion

Left Recursion

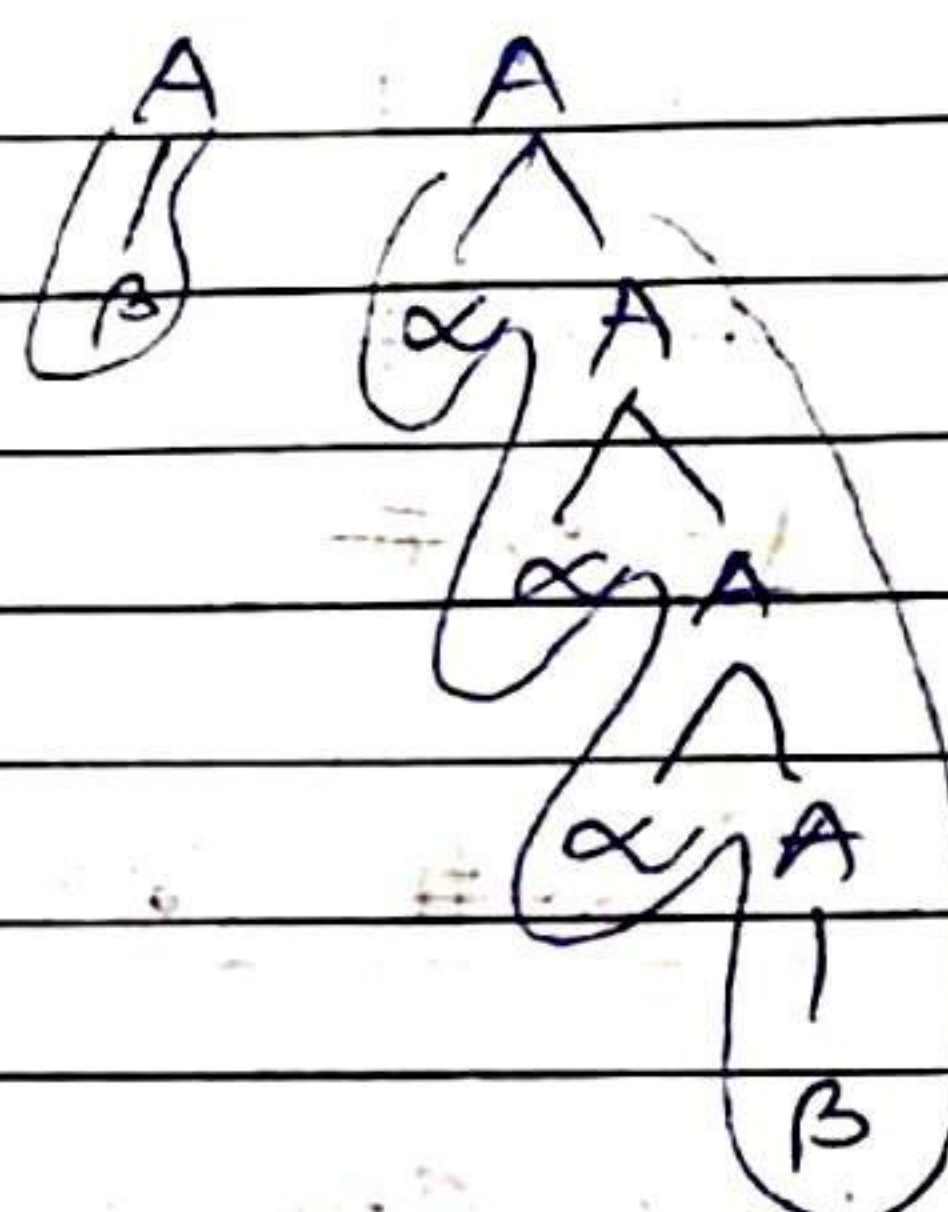
$$A \rightarrow A\alpha / \beta$$



$$L = (\beta\alpha^*)$$

Right Recursion

$$A \rightarrow \alpha A / \beta$$



$$L = (\alpha^*\beta)$$

• $\beta\alpha^*$

$$A \rightarrow \beta\alpha^*$$

$$\begin{matrix} A \rightarrow \beta A' \\ A' \rightarrow \epsilon / \alpha A' \end{matrix} \iff A \rightarrow A\alpha / \beta$$

→ To eliminate left recursion we can follow this above rule.

[Example] - eliminate the left recursive,

$$\begin{matrix} E \rightarrow E + T / T \\ (A) \quad (A) \quad (\alpha) \quad (\beta) \end{matrix}$$

$$\begin{matrix} E \rightarrow TE' \\ E' \rightarrow \epsilon / +TE' \end{matrix}$$

→ This is equivalent grammar and not left recursive grammar.

Example - Eliminate left recursion from this grammar -

$$\textcircled{1} \quad \underline{S} \rightarrow \underline{S} \underline{\alpha} \mid \underline{S} / \underline{\beta}$$

(A) (A) (α) (B)

$$\Rightarrow \begin{cases} S \rightarrow \underline{0} \mid S' \\ S' \rightarrow \epsilon / \underline{0} \underline{S} \underline{S}' \end{cases}$$

$$\textcircled{2} \quad \underline{S} \rightarrow (\underline{L}) / \underline{\alpha}$$

$$\underline{L} \rightarrow \underline{L} \underline{S} / \underline{S}$$

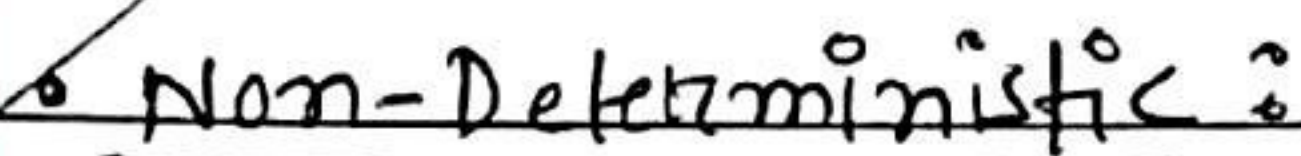
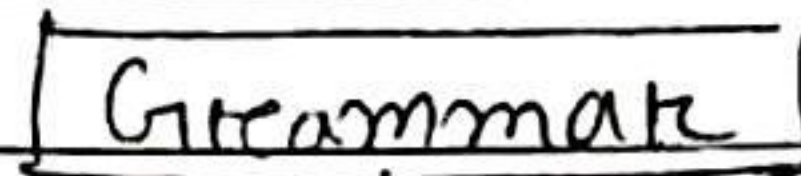
(A) (A) (α) (B)

$$\Rightarrow \begin{cases} S \rightarrow (\underline{L}) / \underline{\alpha} \\ L \rightarrow S L' \\ L' \rightarrow \epsilon / \underline{S} L' \end{cases}$$

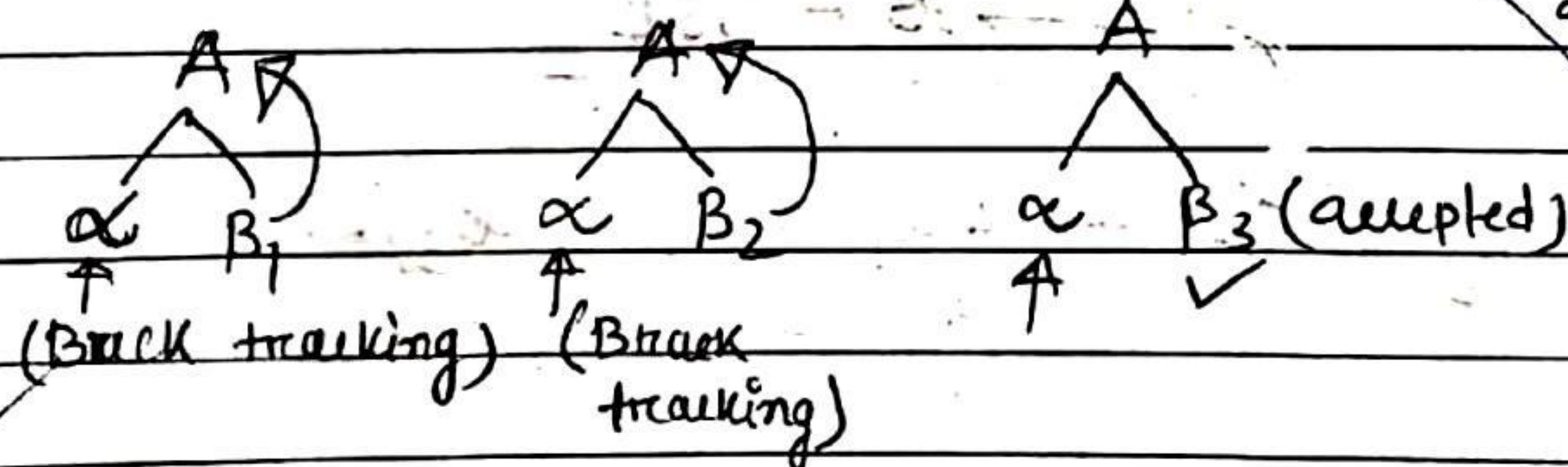
$$\textcircled{3} \quad \underline{A} \rightarrow \underline{A} \underline{\alpha}_1 / \underline{A} \underline{\alpha}_2 / \underline{A} \underline{\alpha}_3 / \dots$$

$$\quad \quad \underline{\beta}_1 / \underline{\beta}_2 / \underline{\beta}_3 / \dots$$

$$\Rightarrow \begin{cases} A \rightarrow \underline{\beta}_1 A' / \underline{\beta}_2 A' / \underline{\beta}_3 A' / \dots \\ A' \rightarrow \underline{\alpha}_1 A' / \underline{\alpha}_2 A' / \underline{\alpha}_3 A' / \dots \end{cases}$$



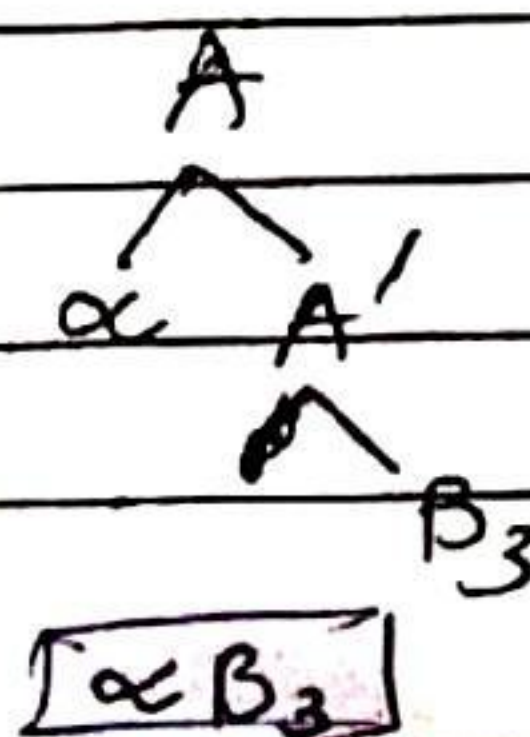
$\alpha \beta_3$



- Left factoring procedure or eliminating non-deter-
-minium:

$$A \rightarrow \alpha A'$$
$$A' \rightarrow \beta_1 / \beta_2 / \beta_3$$

~~generare $\alpha \beta_3$~~



Example -

$$(i) S \rightarrow iEtS$$

$$\quad \quad \quad / iEtseS \rightarrow (\text{Non-deterministic})$$

$$\quad \quad \quad / a$$

$$E \rightarrow b$$

left factoring

$$(ii) S \rightarrow iEtS s' / a$$

$$s' \rightarrow e/es$$

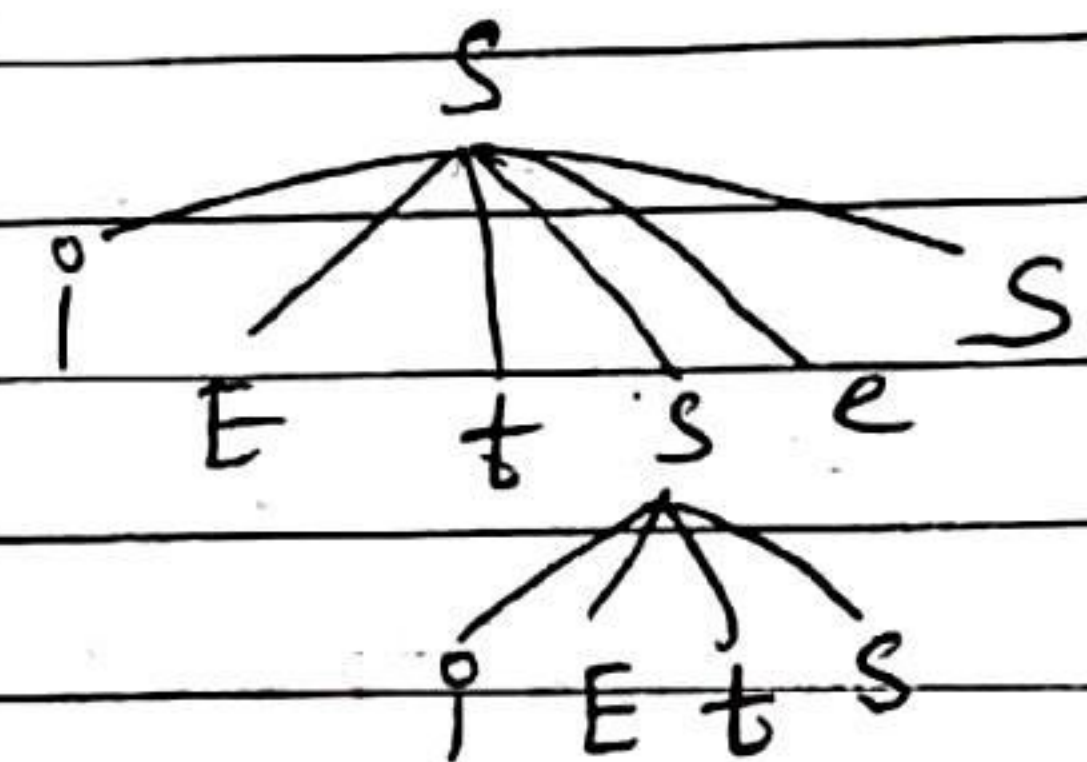
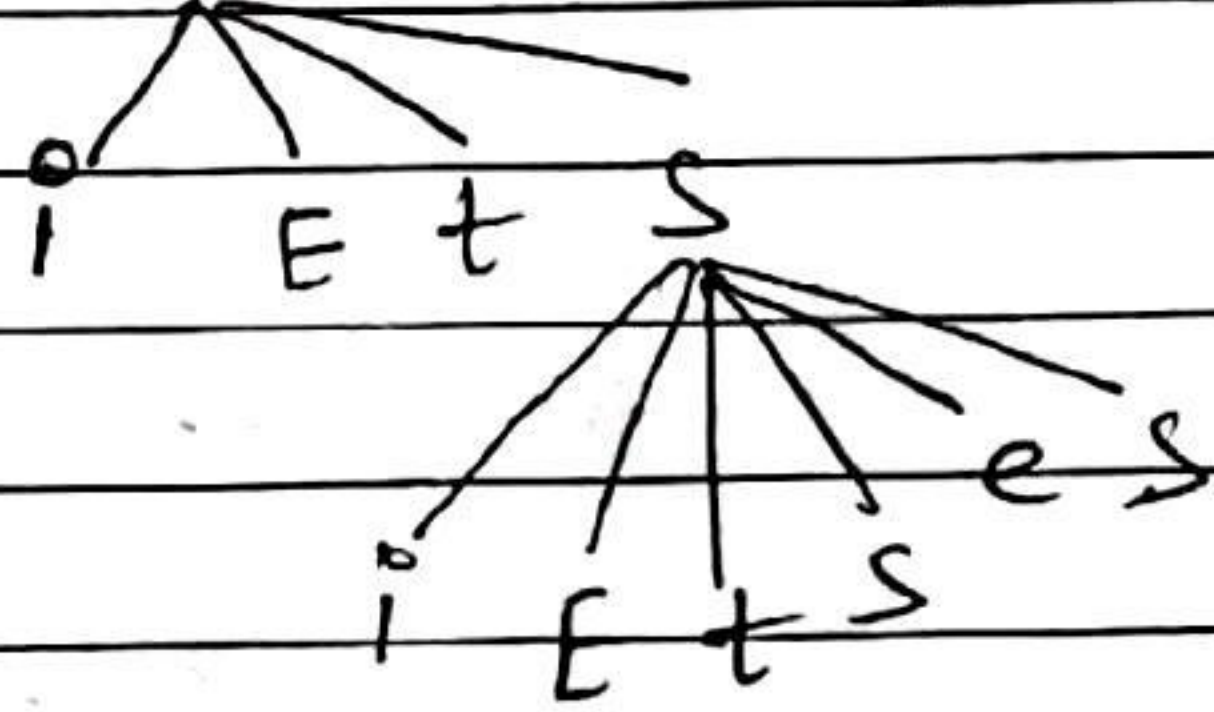
$$E \rightarrow b$$

$$\rightarrow (\text{Deterministic})$$

(i)

S

(W = iEt iEt ses)

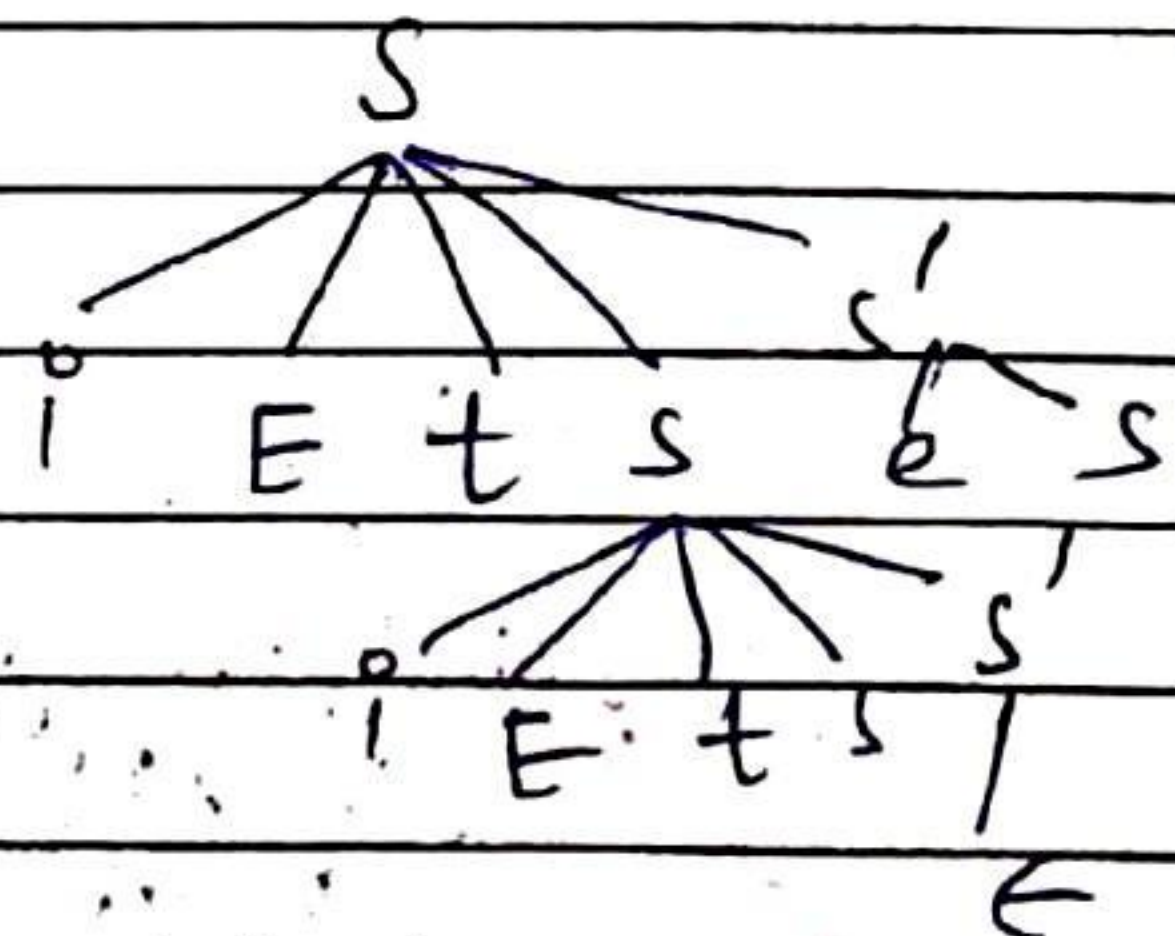
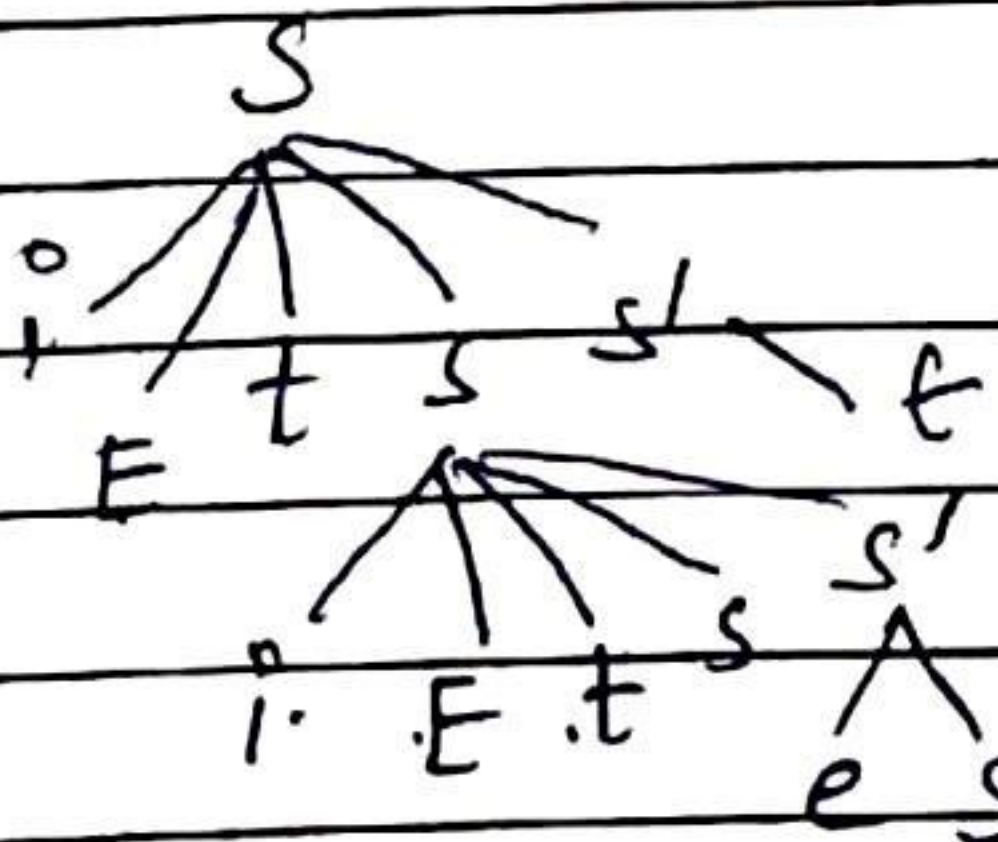


(ambiguous grammar)

→ for the string we get '2' parse tree from the given grammar.

(ii)

W = iEt iEt ses



(also ambiguous grammar)

[Example]-

$s \rightarrow$ a ss b s
 / a s s b s b
 / a b b
 / b

$$\begin{aligned} S &\rightarrow aS' / b \\ S' &\rightarrow (S)bs' / (S)asb \\ &\quad / bb \end{aligned}$$
$$\Rightarrow \begin{array}{l} S \rightarrow aS' / b \\ S' \rightarrow SS'' / bb \\ S'' \rightarrow sbs / asb \end{array}$$

(Deterministic)

$s \rightarrow$

- (b) (c) a a s
- / (b) (s) a s b
- / (b) s b
- / a

$$\Rightarrow \begin{aligned} S &\rightarrow bSS'/a \\ S' &\rightarrow (SaaS)/(Sasb)/b \end{aligned}$$
$$\Rightarrow \begin{array}{l} S \rightarrow bSS'/a \\ S' \rightarrow sSaS''/b \\ S'' \rightarrow aS/sb \\ (\text{Deterministic}) \end{array}$$
